How NIXNAN works: from load to the first HMMA report

Slide-by-slide report accompanying `nixnan-walkthru-tc-rd_nixnan`

Ganesh Gopalakrishnan

2026-10-19

Contents

1 Opening slides **2**
Roadmap . . . 2
Setup, and the trap . . . 2

2 Architecture **3**
Callback architecture . . . 3
Lifecycle of the tool (`nixnan.cu`) . . . 3
First launch of a kernel: what happens . . . 4

3 Encoding **4**
Table-driven: `instructions.json` . . . 4
One 32-bit word per operand: `reginfo` . . . 5
Turning an operand into register reads: `get_regs` . . . 5
Book-keeping on the host: `recorder::mk_entry` . . . 6

4 Injection **6**
Two injected calls per instruction . . . 6

5 Device **7**
`nixnan_check_regs` on the GPU . . . 7
The packet: `exception_info` . . . 7

6 Channel **8**
GPU → CPU channel (NVBit `channel.hpp`) . . . 8
Receiver: `recv` and the thread loop . . . 8
Receiver state machine . . . 9
The "one last message": sentinel flush . . . 9
End of run . . . 9

7 Part A: an ordinary SASS instruction **9**
`HFMA2` in `rd_step_fp16` . . . 9
What came back . . . 10

8 Part B: HMMA **10**
HMMA.16816.F16: one warp instruction, four register groups 10
Which registers are picked up 10
reginfo words for this HMMA 10
What NIXNAN reported (real output, fp16acc16) 11
Summary 11

9 Sentinel / completeness check **11**

10 Histogram pass **12**

Scope and provenance

Everything below comes from reading NIXNAN (branch `print_histo_instrn`, commit `ce35185`): `nixnan.cu`, `inject_funs.cu`, `instruction_info.cu/.cuh`, `instructions.h/.json`, `common.cuh`, `exception_info.cuh`, `fp_utils.cuh`, `recording.cu/.h`, `reginfo.cuh`, and NVBit 1.8's `core/utils/channel.hpp`; plus real runs of `rd_nixnan` (scalar) and `tensor_core/rd_tc` on an RTX 3090, CUDA 12.4. Raw logs: `latex/data/` (Parts A and B) and `tensor_core/logs/`. Statements marked *(my reading)* are inferences from the code, not observations.

This report follows the deck slide by slide, in order. Each section gives the slide title, a prose explanation, the source excerpt behind it, and real output where the slide shows any.

1 Opening slides

Slide: Roadmap

The deck has three movements. First the *machinery common to every instrumented instruction* (setup, callbacks, lifecycle, first-launch instrumentation, the compact encoding, the two injected calls, the device function, the channel, the receiver and the final report). Then **Part A**, one ordinary SASS instruction (an `HFMA2` from the scalar FP16 reaction–diffusion kernel). Then **Part B**, one tensor-core instruction (`HMMA.16816.F16` from `rd_tc`). The point of the order is that HMMA needs *no special machinery*: it is one more table entry in `instructions.json` whose operand groups happen to be wide.

Slide: Setup, and the trap

All runs use

```
env -u LD_LIBRARY_PATH LINE_INFO=1 PRINT_ILL_INSTR=1 TOOL_VERBOSE=1 \
  LOGFILE=latex/data/partA-scalar.log \
  LD_PRELOAD=~/repos/nixnan/nixnan.so ./rd_nixnan
```

`LD_PRELOAD` makes the dynamic loader map `nixnan.so` (which statically contains the NVBit core) before the CUDA runtime, so NVBit can intercept every driver call. Two things went wrong on this machine and are worth telling an audience.

1. A driver upgrade without a reboot left the kernel module at 595.84 while `libcuda` was 595.99.02. Plain CUDA programs still ran, but NVBit never saw a kernel launch (even NVBit's own `instr_count` sample printed nothing). A reboot fixed it.

2. After the reboot, NIXNAN still printed only its banner until `LD_LIBRARY_PATH` (which contained a nonexistent `/usr/local/cuda-13.2/lib64` and a dyninst directory) was removed with `env -u`. The same binary then reported. I did not isolate which entry matters.

2 Architecture

Slide: Callback architecture

NVBit sits between the application and the driver. The tool exports four callbacks, which NVBit calls:

```
void nvbit_at_init();                          // before any CUDA call
void nvbit_tool_init(CUcontext ctx);           // context created
void nvbit_at_cuda_event(CUcontext ctx, int is_exit, nvbit_api_cuda_t cbid,
                         const char *name, void *params, CUresult *pStatus);
void nvbit_at_ctx_term(CUcontext ctx);         // context destroyed
```

`nvbit_at_cuda_event` is invoked twice around every driver call (`is_exit` = 0 then 1). NIXNAN only reacts to the launch APIs: `cuLaunchKernel`, its `_ptsz` form, and the cooperative variants.

Slide: Lifecycle of the tool (`nixnan.cu`)

`nvbit_at_init`. Reads configuration with `GET_VAR_INT/STR`, which also builds the banner text seen at the top of every log. It sets `CUDA_MANAGED_FORCE_DEVICE_ALLOC=1`, because `channel_dev` is declared `static __managed__`.

```
setenv("ACK_CTX_INIT_LIMITATION", "1", 1);
setenv("CUDA_MANAGED_FORCE_DEVICE_ALLOC", "1", 1);
GET_VAR_INT(instr_begin_interval, "INSTR_BEGIN", 0, ...);
GET_VAR_INT(sampling, "SAMPLING", 0, ...);
GET_VAR_INT(line_info, "LINE_INFO", 1, ...);
```

`nvbit_tool_init`. Prints "Initializing GPU context…", builds the recorder, the channel and the receiver thread:

```
recorder = std::make_shared<nixnan::recorder>(TABLE_SIZE);   // 1<<17
recv_thread_started = true;
channel_host.init(0, CHANNEL_SIZE, &channel_dev, NULL);
recv_thread = std::thread(recv_thread_fun, recorder, channel_host);
```

Note that `channel_host.init` is given `NULL` as thread function: NVBit's own receiver thread is not used; NIXNAN runs its own `std::thread`. Also `CHANNEL_SIZE = sizeof(exception_info)`, so the buffer size passed to the channel is one packet's size (see the channel slides).

`nvbit_at_cuda_event`, entry. Sets `recv_thread_receiving = true`, calls `instrument_function`, prints "running kernel […]" the first time a kernel name is seen, and calls `nvbit_enable_instrumented(ctx, f, sampling>0 ? count%sampling==0 : true)`.

Exit. `cudaDeviceSynchronize()` and a `cudaGetLastError()` check (see the sentinel slide for what was removed).

`nvbit_at_ctx_term`. Stops and joins the receiver, copies the counters back, prints the report.

Real output. The head of a real run:

```
#nixnan: Initializing GPU context...
#nixnan: Instrumenting all functions
#nixnan: running kernel [rd_tc_fp16acc16] ...
```

Slide: First launch of a kernel: what happens

`instrument_function` is the heart of the tool. Its steps:

1. `nvbit_get_related_functions(ctx, kernel)` returns device functions the kernel can call; the kernel itself is appended.
2. A set `instrumented_functions` (`insert(f).second`) makes sure each function is instrumented exactly once; later launches skip to step 6.
3. `should_instrument` applies the whitelist/blacklist (and is false when `LOG_KERNELS` is active).
4. For every `Instr*` from `nvbit_get_instrs(ctx, f)`: `get_reginfo(instr)`; if it is empty and the instruction is not a memory instruction, skip.
5. Otherwise `recorder->mk_entry` allocates an `inst_id` and two `nvbit_insert_call`s are made (next slides).
6. Back in the callback, `nvbit_enable_instrumented` switches the instrumented copy on or off for this launch. With `SAMPLING=k`, only every k-th launch of a kernel runs the instrumented code.

```
for (auto instr : nvbit_get_instrs(ctx, f)){
  auto reg_infos = instruction_info::get_reginfo(instr);
  bool meminstr = is_memory_instruction(instr);
  if (reg_infos.empty() && !meminstr) { continue; }
  ...
```

The verbose log shows this loop at work, one "Instrumenting instruction" per accepted SASS instruction; instructions with no table entry never appear (and `STG.E.U16` appears with "Unsupported store instruction" since `INSTR_MEM` is off):

```
#nixnan: Inspecting function rd_step_fp16(__half const*, __half*, __half, __half) at address 0
    x7271d7371300
#nixnan: Instrumenting instruction @!P0 STG.E.U16 [R2.64], RZ ;
#nixnan: Unsupported store instruction: STG.E.U16
#nixnan: Instrumenting instruction HADD2 R0, R0.H0_H0, R7.H0_H0 ;
```

3 Encoding

Slide: Table-driven: `instructions.json`

The only per-opcode knowledge in NIXNAN is a JSON table, embedded in the binary as a byte array (`instructions.h`, generated from the `.json`) and parsed once with nlohmann::json. For each operand position of the SASS instruction it says which floating-point format lives there and how many values:

```
"HMMA.16816.F16": {"registers": [
   {"type":"f16","count":4},   // operand 0  D
   {"type":"f16","count":8},   // operand 1  A
   {"type":"f16","count":4},   // operand 2  B
   {"type":"f16","count":4} ]} // operand 3  C
```

Keys are tried as *regular expressions* against the opcode string (`std::regex_match`), the first match wins; so a single key such as `"(FFMA(?:\\.FTZ)?(?:\\.(RP|RM|RZ))?(?:\\.SAT)?)"` covers all modifier combinations.

```
for (auto it = instructions.begin(); it != instructions.end(); ++it) {
    std::regex re(it.key());
    if (std::regex_match(inst_name, re)) { matched_key = it.key(); break; }
}
```

`count` is the number of *values* (not registers), converted by

```
case FP16: case BF16: return (count + 1) / 2;   // packed pairs
case FP32: return count;
case FP64: return 2 * count;
```

Opcodes without an entry are silently ignored. I verified that `F2FP.PACK` is one of them (`grep -c F2FP instructions.json` gives 0), which explains why no FP32→FP16 conversion is ever reported (see Part B).

Slide: One 32-bit word per operand: `reginfo`

Each operand is described to the GPU by one packed 32-bit word:

```
struct reginfo {
    int count: 6;  int32_t type: 4;
    bool half_h0: 1; bool half_h1: 1; bool div0: 1;
    unsigned int operand: 2;  int num_regs: 4;  bool f64high: 1;
private: int32_t reserved: 12;
};
static_assert(sizeof(reginfo) == sizeof(int32_t));
```

Because it is exactly 32 bits it is passed with `nvbit_add_call_arg_const_val32(instr, tobits32(ri), true)` as one call argument. `type` codes (`common.cuh`): FP16=0, BF16=1, FP32=2, FP64=3. `OPERANDS=4` because `operand` has 2 bits. Observation: `num_regs` is a *signed* 4-bit field while `get_reginfo` asserts only `get_num_regs<16`, so values ≥ 8 would not round-trip (my reading; no instruction in the table needs more than 4).

Slide: Turning an operand into register reads: `get_regs`

`get_regs` looks at the actual SASS operand and returns a vector of *deferred lambdas* (`reginsertion`) which will each add one call argument:

```
case OperandType::REG: case OperandType::UREG: {
    size_t reg_start = op->u.reg.num - (f64high ? 1 : 0);
    for (size_t i = 0; i < num_regs; i++)
        reg_ops.push_back([instr, reg_start, i, type]() {
            auto fn = type == OperandType::REG ? nvbit_add_call_arg_reg_val
                                               : nvbit_add_call_arg_ureg_val;
            fn(instr, clamp(reg_start+i), true); });
    break; }
case OperandType::CBANK: ... nvbit_add_call_arg_cbank_val(instr, cbank.id, offset, true);
```

Registers are consecutive: `R4` with `num_regs`=4 means R4,R5,R6,R7. `clamp` caps the index at 255, which is `RZ` (the zero register); this is presumably how the `RZ` accumulator in the first HMMA of each chain (`HMMA.16816.F16 R8, R24.reuse, R8, RZ`) is read, as zero (my reading; not tested). Immediates are examined on the *host*, once; a NaN, Inf or subnormal immediate produces a warning and no runtime argument.

The lambdas are deferred because NVBit requires the call's arguments in order; the fixed arguments are pushed first, then the lambdas run (the `for (auto& rfun : rfuns) rfun();` loop in `instrument_function`).

Slide: Book-keeping on the host: `recorder::mk_entry`

Strings never go to the GPU. `mk_entry` stores them host-side under a dense id:

```
inst_info[current_entry].instr = instr;   // instr->getSass()
inst_info[current_entry].path  = path;    // from nvbit_get_line_info
inst_info[current_entry].line  = line;
inst_info[current_entry].func  = func;    // cut_kernel_name(...)
std::memcpy(inst_info[current_entry].opertypes, optypes, OPERANDS);
return current_entry++;
```

Source locations are fetched only when `LINE_INFO=1`, under a temporary `SIGSEGV` handler that prints "A segmentation fault occurred in NVBit while trying to access debug information. Please try with the LINE_INFO variable set to 0." and exits. Device counters: an array of $2^{17} \cdot 2^7$ `uint32` cleared at construction, indexed by

```
index = id << SH_AMT | op << EXCEBITS | exce;     // SH_AMT = 5 + 2
```

so each (instruction, operand, flag-set) has its own counter, with the five exception bits `E_NAN=1, E_INF=2, E_NINF=4, E_SUB=8, E_DIV0=16` as the low index bits.

4 Injection

Slide: Two injected calls per instruction

For each accepted instruction NIXNAN adds *two* calls to the same device function:

```
nvbit_insert_call(instr, "nixnan_check_regs", IPOINT_AFTER);
nvbit_add_call_arg_guard_pred_val(instr);
nvbit_add_call_arg_const_val64(instr, tobits64(recorder->get_device_recorder()), false);
nvbit_add_call_arg_const_val32(instr, inst_id, false);
nvbit_add_call_arg_const_val64(instr, tobits64(&channel_dev), false);
nvbit_add_call_arg_const_val32(instr, 1 + rfuns.size());     // arg_count
nvbit_add_call_arg_const_val32(instr, tobits32(ri), true);   // operand 0
for (auto& rfun : rfuns) rfun();
```

The AFTER call checks **operand 0, the destination**: the result the instruction just wrote. The BEFORE call checks **operands 1...n, the sources**, with `arg_count = sum(num_regs) + (#operands - 1)`, i.e. reginfo words plus register values. Reading sources before is essential when an operand is both source and destination, e.g. `HMMA R26, R4, R30, R26`: after execution R26 holds D, so only a BEFORE read sees the accumulator input C. The guard predicate (`@!P0`-style) is passed so predicated-off threads do nothing.

5 Device

Slide: `nixnan_check_regs` on the GPU

```
extern "C" __device__ __noinline__ void
nixnan_check_regs(int pred, device_recorder recorder, uint32_t inst_id,
                  ChannelDev* pchannel_dev, uint32_t arg_count, ...) {
  if (!pred) return;
  int active_mask = __ballot_sync(__activemask(), 1);
  const int first_laneid = __ffs(active_mask) - 1;
  ...
  while (arg_count > 0) {
    reginfo reg_info = va_arg(ap, reginfo); arg_count--;
    for (int j = 0; j < reg_info.count; j++) {
      switch (reg_info.type) {
        case FP16: { uint32_t val = va_arg(ap, uint32_t);
            exces[operand] |= half2_classify(val, h0, h1, div0);
            arg_count--; j++; break; }
        case FP32: ... float_classify(val, div0) ...
        case FP64: low, high -> double_classify
```

Variadic arguments carry the register values. FP16 and BF16 registers hold two values each, so a register consumes two of `count` (hence `j++`). The classification helpers (`fp_utils.cuh`) test the bit patterns directly, e.g. for single precision exponent 0xFF with mantissa $\neq 0$ is NaN, with mantissa 0 is Inf; exponent 0 with mantissa $\neq 0$ is subnormal. Each operand's result goes into `exces[operand]`. Then:

```
for (int op = 0; op < OPERANDS; op++)
  for (int tid = 0; tid < 32; tid++)
    exces[op] |= __shfl_sync(active_mask, exces[op], tid);
if (first_laneid == laneid)
  for (int op = 0; op < OPERANDS; op++) {
    uint32_t exce = exces[op];
    if (exce != 0) {
      uint32_t num_exceptions = recorder.record(inst_id, exce, op);  // atomicAdd
      if (num_exceptions == 0) {                // first time only
        for (int skip = 0; skip < 2; skip++) {
          exception_info ei(get_ctaid(), get_warpid(), inst_id, exce, op, UNKNOWN, skip);
          channel_dev->push(&ei, sizeof(exception_info)); } } } }
```

So a warp's 32 lanes are merged into one verdict, one lane does the bookkeeping, and only the *first* sighting of an (instruction, operand, flag-set) triple crosses to the CPU, in two packets. Everything later is just an `atomicAdd`; that is the "repeats" count in the final report.

Slide: The packet: `exception_info`

```
struct exception_info {      // exception_info.cuh
  int4 cta;                  // x,y,z of the block; .w = warp id
  uint32_t inst_id, exce, oper, tp_override;
  bool skip = false; };
```

(The constructor assigns `this->cta = cta; cta.w = warp_id;` — the second assignment writes to the *parameter*, not the member, so the stored `.w` is whatever the caller's `cta.w` was. For a real packet `get_ctaid()` supplies it; for the sentinel `int4{-1,-1,-1,-1}` gives `warp()==-1`. This is my reading of the code, not something I tested.) The receiver converts bit flags back into words and decodes the type from the recorder unless `tp_override` is not `UNKNOWN`.

6 Channel

Slide: GPU → CPU channel (NVBit `channel.hpp`)

The device-side `push`:

```
curr_ptr = (uint8_t*)atomicAdd((ULL*)&buff_write_head_ptr, (ULL)nbytes);
if (curr_ptr + nbytes > buff_end) {
    if (curr_ptr <= buff_end) {                  // first to see "full"
        while (buff_write_tail_ptr != curr_ptr) {}   // wait for writers
        flush();
    } else { while (buff_write_head_ptr > buff_end) {} }   // wait for the flush
    curr_ptr = NULL; }
memcpy(curr_ptr, packet, nbytes);
atomicAdd((ULL*)&buff_write_tail_ptr, (ULL)nbytes);
```

and `flush`:

```
uint32_t nbytes = (uint32_t)(buff_write_tail_ptr - buff);
if (nbytes == 0) return;
__threadfence_system();      // make the pinned-memory writes visible
*doorbell = nbytes;          // notify the host
while (*doorbell != 0) ;     // wait for the host to release
buff_write_tail_ptr = buff; __threadfence(); buff_write_head_ptr = buff;
```

The buffer is pinned host memory (`cudaHostAlloc`), the doorbell a mapped host `int` which the GPU sees through `cudaHostGetDevicePointer`. **Capacity.** Since NIXNAN passes `buff_size = sizeof(exception_info)`, the buffer holds exactly one packet: after any push the next push finds the buffer full and triggers a flush (my reading from `CHANNEL_SIZE` and `init`; I did not instrument it). That is why the two-packet `skip` pair matters (see below).

Slide: Receiver: `recv` and the thread loop

```
uint32_t buff_nbytes = *doorbell;  if (buff_nbytes == 0) return 0;
int nbytes = min(buff_nbytes, max_buff_size);
memcpy(buff, dev_buff_read_head, nbytes);
int bytes_left = buff_nbytes - nbytes;
if (bytes_left > 0) dev_buff_read_head += nbytes; else dev_buff_read_head = dev_buff;
*doorbell = bytes_left;          // 0 releases the GPU
```

The doorbell therefore doubles as a byte counter: each call returns at most one packet and writes back the remainder; the spinning GPU is released only when it reaches 0. The loop (`recv_thread_fun`) is a busy poll:

```
while (recv_thread_started) {
  if (recv_thread_receiving &&
      (num_recv_bytes = channel_host.recv(recv_buffer, CHANNEL_SIZE)) > 0) {
     ... for each packet:
       if (ei->to_skip()) { num_processed_bytes += sizeof(exception_info); continue; }
       if (ei->warp() == -1) { recv_thread_receiving = false; break; }
       ... build string; nnout_line(oss.str());
```

`nnout_line` takes a mutex and flushes, so lines from the exception thread and the histogram thread do not interleave (commit `2054a32`). With `MAX_ERRORS=n` the thread calls `nnterminate`.

Slide: Receiver state machine

States: *idle* (`recv_thread_receiving=false`) → *polling* (set at kernel launch) → *decode* on bytes > 0; `skip` packets step over; a sentinel returns to idle. About `skip`: every first sighting is pushed twice, `skip=0` then `skip=1`, and the receiver acts on the first and steps over the second. The intent is not documented in the repository. Reading `channel.hpp` together with `CHANNEL_SIZE = sizeof(exception_info)` gives a concrete explanation (my reading): the buffer holds exactly one packet and `push` flushes only when the *next* packet does not fit, so a lone packet would sit unflushed; the second, dummy packet (`skip=1`) forces the real one out and is ignored by the receiver. The last dummy always remains in the buffer. Section 9 shows the observed consistency.

Slide: The "one last message": sentinel flush

The design:

```
__global__ void flush_channel() {
  exception_info ei(int4{-1, -1, -1, -1}, 0, 0, 0, 0);   // warp() == -1
  channel_dev.push(&ei, sizeof(exception_info));
  channel_dev.flush();
}
```

launched after each kernel with `skip_flag` protecting against re-entering the launch callback, followed by `while (recv_thread_receiving) sched_yield();`. In commit `ce35185` *all of this is commented out* in `nvbit_at_cuda_event`; only `cudaDeviceSynchronize()` and the error check remain. The receiver still honours `warp()==-1`, but nothing sends it, so `recv_thread_receiving` stays true from the first launch on. Section 9 reports what I could test.

Slide: End of run

```
recv_thread_started = false;  recv_thread.join();
recorder->end();             // cudaMemcpy device_errors -> host_errors
```

then for every (instr, op, flag-set) with $n > 0$ hits it adds n to the *repeats+distinct* sum for each flag in the set, and 1 to the distinct-location count; the printed "repeats" is total minus distinct. A real report (Part B, fp16acc16):

```
#nixnan: ------------ nixnan Report -----------
#nixnan: --- FP16 Operations ---
#nixnan: NaN:                    50 (722 repeats)
#nixnan: Infinity:               20 (48 repeats)
```

7 Part A: an ordinary SASS instruction

Slide: `HFMA2` in `rd_step_fp16`

From the verbose log (`latex/data/partA-scalar.log`):

```
#nixnan: Instrumenting instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].H0_H0, R9.H0_H0 ;
#nixnan: Instrumenting instruction with 2 registers
#nixnan: Instrumenting operand 0. div0: 0, regs: 1, count: 2
#nixnan: Instrumenting instruction with 6 registers
#nixnan: Instrumenting operand 1. div0: 0, regs: 1
#nixnan: Instrumenting operand 2. div0: 0, regs: 1
#nixnan: Instrumenting operand 3. div0: 0, regs: 1
```

Operand 0 (D=R0) is one 32-bit register holding two halves; AFTER has arg_count=2 (one reginfo + one value). The BEFORE call has three reginfo words and three values = 6. Operand 2 is a constant-bank operand c[0x0][0x170], passed with nvbit_add_call_arg_cbank_val. The ".H0_H0" suffix on the SASS operand means the low half is replicated into both lanes; NIXNAN ignores it and inspects both halves of the register (half_h0=half_h1=true), which is a conservative approximation (my reading).

Slide: What came back

Real report lines for this instruction:

```
#nixnan: error [infinity] detected in operand 1 of instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].
    H0_H0, R9.H0_H0 ; in function rd_step_fp16 of type f16
#nixnan: error [infinity] detected in operand 0 of instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].
    H0_H0, R9.H0_H0 ; in function rd_step_fp16 of type f16
```

Operand 1 is checked BEFORE (the previous step's inf arrives as an input), operand 0 AFTER (the result). No at file:line: rd_nixnan is built without -lineinfo. The type text f16 is looked up from the recorder, because the packet's tp_override is UNKNOWN.

8 Part B: HMMA

Slide: HMMA.16816.F16: one warp instruction, four register groups

SASS (cuobjdump -sass rd_tc): HMMA.16816.F16 R26, R4, R30, R26 ;. Verbose log for HMMAs of this kernel (latex/data/partB-hmma.log):

```
#nixnan: Instrumenting instruction HMMA.16816.F16 R22, R12.reuse, R30, R22 ;
#nixnan: Instrumenting instruction with 3 registers
#nixnan: Instrumenting operand 0. div0: 0, regs: 2, count: 4
#nixnan: Instrumenting instruction with 11 registers
#nixnan: Instrumenting operand 1. div0: 0, regs: 4
#nixnan: Instrumenting operand 2. div0: 0, regs: 2
#nixnan: Instrumenting operand 3. div0: 0, regs: 2
```

3 = 1 reginfo + 2 values (AFTER); 11 = 3 reginfo + 4 + 2 + 2 values (BEFORE).

Slide: Which registers are picked up

For HMMA.16816.F16 R26, R4, R30, R26: D (op 0, 4 halves) → R26,R27 read AFTER; A (op 1, 8 halves) → R4–R7; B (op 2, 4 halves) → R30,R31; C (op 3, 4 halves) → R26,R27, all BEFORE. Thus the old accumulator is seen even though R26 is overwritten. The $m16n8k16$ half-precision fragment layout per thread (A: 8 halves, B: 4, C/D: 4) is why the counts are 8/4/4/4. Each of the 32 lanes supplies its own registers; the device function ORs the 32 verdicts.

Slide: reginfo words for this HMMA

Produced by compiling the real reginfo.cuh with g++ and filling the fields as get_reginfo does:

operand	count	num_regs	word
0 (D)	4	2	`0x00010C04`
1 (A)	8	4	`0x00022C08`
2 (B)	4	2	`0x00014C04`
3 (C)	4	2	`0x00016C04`

They match my earlier hand computation. For the `.F32` variant (type=2): D = `0x00020C84`, C = `0x00026C84`. These words were *not* dumped from the running tool (it leaves the 12 reserved bits uninitialised; the device code never reads them).

Slide: What nixnan reported (real output, fp16acc16)

```
#nixnan: error [infinity] detected in operand 0 of instruction HMMA.16816.F16 R26, R4, R30, R26 ;
    in function rd_tc_fp16acc16 of type f16
#nixnan: error [infinity] detected in operand 0 of instruction HMMA.16816.F16 R4, R8.reuse, R4, R22
     ; ...
#nixnan: error [infinity] detected in operand 3 of instruction HMMA.16816.F16 R6, R8, R6, R26 ; ...
```

Operand 0: the tensor core produced the overflow in its f16 accumulator (step 230). The step after, inf inputs meet the zeros in P: $0 \times \infty = \text{NaN}$ inside the HMMA, and the NaN lines appear (operands 0–3 of the HMMAs). With `fp16acc32/bf16acc32` the first inf is born in `F2FP.PACK` / `F2FP.BF16.PACK`, which has no table entry, so NIXNAN shows it first as an operand of the next HMMA. `LINE_INFO=1` gave no `mma_sync` source lines in these runs.

Slide: Summary

The tool is a table (opcode → operand groups), one 32-bit `reginfo` per operand, two injected calls (destination AFTER, sources BEFORE), a warp-merged check with per-site atomic counters, a first-occurrence-only channel, and a single polling receiver. HMMA required no new code, only a table entry.

9 Sentinel / completeness check

I could not and did not modify the NIXNAN sources, so I did not run a variant with the sentinel re-enabled. What I did is a consistency check on the three `tensor_core/logs/nixnan-*.log`: every first-occurrence packet that reached the receiver was printed live, and the final table is computed from the device counters (`device_errors`), which do *not* depend on the channel. If any packet had been lost, the table would count more distinct (instruction, operand, flag-set) triples than there are live lines. They agree:

run	live lines	NaN live	NaN report	Inf live	Inf report
fp16acc16	68	52	50 (f16) + 2 (mem)	20	20 (f16)
fp16acc32	58	50	16 (f16) + 34 (f32)	12	8 + 4
bf16acc32	82	66	24 (bf16) + 42 (f32)	36	16 + 20

A live line containing "NaN,infinity" counts toward both columns, as does the report. The 2 extra NaN lines in fp16acc16 are `STG.E` stores, reported in the memory section of the table. Conclusions, with their limits. (1) In these three runs no first-occurrence report was lost. (2) That is consistent

with the dummy-packet reading of `skip`, but I did not test it directly. (3) What can still be lost: the trailing dummy packet (harmless), and nothing else under my reading; a variant where the last *real* packet is not followed by a dummy cannot occur, because both are pushed together by the same lane. (4) The sentinel/wait loop being disabled means `recv_thread_receiving` is never cleared; the receiver simply polls until context teardown.

10 Histogram pass

(to be filled)